

百度之星 / 算法竞赛常用算法总结

算法流程、Python 解题模板与题型识别

整理版

目录

1 总览：如何识别题型	4
1.1 常见题型与算法映射	4
1.2 Python 竞赛基本框架	4
1.3 复杂度速查	4
2 基础算法	5
2.1 排序与自定义排序	5
2.2 前缀和	5
2.3 二维前缀和	5
2.4 差分数组	6
2.5 离散化	6
2.6 二分查找	7
2.7 二分答案	7
3 双指针、滑动窗口与单调结构	7
3.1 双指针	7
3.2 滑动窗口	8
3.3 单调栈	8
3.4 单调队列	8
4 贪心	9
4.1 贪心的判断方法	9
4.2 区间调度	9
4.3 区间合并	9
5 数据结构	10
5.1 堆	10
5.2 懒删除堆	10
5.3 并查集	10
5.4 树状数组	11
5.5 线段树	12
5.6 ST 表	13
6 图论	13
6.1 建图模板	13
6.2 BFS	14
6.3 DFS	14

6.4	拓扑排序	14
6.5	Dijkstra	15
6.6	Floyd	15
6.7	最小生成树 Kruskal	16
7	动态规划	16
7.1	DP 设计四步	16
7.2	一维线性 DP	16
7.3	最长上升子序列	16
7.4	01 背包	17
7.5	完全背包	17
7.6	区间 DP	17
7.7	树形 DP	17
7.8	状态压缩 DP	18
7.9	数位 DP	18
8	数学与数论	19
8.1	最大公约数与最小公倍数	19
8.2	快速幂	19
8.3	逆元	19
8.4	组合数预处理	19
8.5	素数筛	20
8.6	质因数分解	20
8.7	矩阵快速幂	21
9	字符串算法	21
9.1	KMP	21
9.2	Trie 字典树	22
9.3	字符串哈希	22
9.4	Manacher	23
10	搜索、回溯与剪枝	23
10.1	回溯模板	23
10.2	Meet in the Middle	23
11	位运算	24
11.1	常用技巧	24
11.2	枚举子集	24
12	扫描线与区间覆盖	25
12.1	最大重叠区间数量	25
12.2	整数闭区间覆盖	25
13	博弈论	25
13.1	Nim 游戏	25
13.2	SG 函数	26

14 常见综合题型	26
14.1 最值 + 可行性	26
14.2 区间修改 + 区间查询	26
14.3 连通块问题	26
14.4 计数取模	27
14.5 字符串匹配	27
15 比赛实用技巧	27
15.1 读题顺序	27
15.2 Python 常见坑	27
15.3 浮点二分模板	28
15.4 常用取模写法	28
16 赛前复习清单	28
17 最后的选型口诀	28

1 总览：如何识别题型

1.1 常见题型与算法映射

题面信号	优先考虑	典型关键词
求区间和、区间次数、区间最值	前缀和、差分、树状数组、线段树、ST 表	多次询问、区间修改、静态 RMQ
最小化最大值、最大化最小值	二分答案	最少时间、最大距离、最小代价、能否完成
连续子数组、双端移动	双指针、滑动窗口、前缀和 + 哈希	最长、最短、恰好/至少/至多
维护动态集合最值	堆、平衡思想、懒删除	第 k 大、实时中位数、优先级
连通性、合并集合	并查集、DFS/BFS	联通块、朋友关系、合并、判断同组
最短路、路径代价	Dijkstra、BFS、Floyd	路径、代价、距离、边权
有向依赖关系	拓扑排序、DAG DP	先后顺序、课程、任务依赖
选择若干物品最优	背包 DP、贪心、排序	容量、价值、收益、限制
字符串匹配	KMP、哈希、Trie、Z 函数	子串、前缀、模式串、字典树
取模计数	组合数学、快速幂、DP	$10^9 + 7$ 、方案数、排列组合
博弈轮流操作	SG 函数、奇偶性、Nim	两人轮流、必胜、必败
坐标很大但点少	离散化、扫描线	坐标到 10^9 、事件、覆盖

1.2 Python 竞赛基本框架

```

import sys
from collections import deque, defaultdict, Counter
from heapq import heappush, heappop
from bisect import bisect_left, bisect_right
from math import gcd, isqrt

input = sys.stdin.readline
MOD = 10 ** 9 + 7
INF = 10 ** 30

def solve():
    n = int(input())
    a = list(map(int, input().split()))
    # write solution here

if __name__ == "__main__":
    solve()
    
```

1.3 复杂度速查

- $n \leq 20$: 可考虑状态压缩、枚举子集、Meet in the Middle。
- $n \leq 10^3$: 可考虑 $O(n^2)$ DP 或图算法。
- $n \leq 10^5$: 一般需要 $O(n \log n)$ 或 $O(n)$ 。

- $n \leq 10^6$: 一般需要 $O(n)$, 并注意输入输出效率。
- 多组测试时, 总复杂度看 $\sum n$, 不要只看单组 n 。

2 基础算法

2.1 排序与自定义排序

流程:

1. 找出比较关键字。
2. 判断升序还是降序。
3. 若有多个条件, 使用元组排序。

适用题型: 区间排序、贪心选择、按结束时间排序、按收益排序、事件扫描。

```
a.sort()                # ascending
a.sort(reverse=True)    # descending
items.sort(key=lambda x: (x[0], -x[1]))
```

2.2 前缀和

核心思想: 用 $pre[i]$ 表示前 i 个元素之和, 则区间 $[l, r]$ 的和为

$$sum(l, r) = pre[r + 1] - pre[l].$$

算法流程:

1. 建立长度为 $n + 1$ 的前缀数组。
2. $pre[i + 1] = pre[i] + a[i]$ 。
3. 查询时用差值快速得到区间信息。

适用题型: 多次区间求和、区间统计、子数组和、二维矩阵区域和。

```
def prefix_sum(a):
    pre = [0]
    for x in a:
        pre.append(pre[-1] + x)
    return pre

def range_sum(pre, l, r):
    return pre[r + 1] - pre[l]
```

2.3 二维前缀和

二维前缀和适合矩阵子矩形求和。

$$S[x_2][y_2] - S[x_1][y_2] - S[x_2][y_1] + S[x_1][y_1]$$

```
def build_2d_prefix(mat):
    n, m = len(mat), len(mat[0])
    s = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n):
        row_sum = 0
        for j in range(m):
            row_sum += mat[i][j]
            s[i + 1][j + 1] = s[i][j + 1] + row_sum
    return s

def rect_sum(s, x1, y1, x2, y2):
    # rectangle [x1, x2) * [y1, y2)
    return s[x2][y2] - s[x1][y2] - s[x2][y1] + s[x1][y1]
```

2.4 差分数组

核心理想：若要对区间 $[l, r]$ 加 v ，只需：

$$diff[l] += v, \quad diff[r + 1] -= v.$$

最后对差分数组做一次前缀和即可还原。

适用题型：多次区间加、最终数组、区间覆盖次数、离线修改。

```
def apply_range_add(n, ops):
    diff = [0] * (n + 1)
    for l, r, v in ops:
        diff[l] += v
        if r + 1 < n:
            diff[r + 1] -= v
    ans = [0] * n
    cur = 0
    for i in range(n):
        cur += diff[i]
        ans[i] = cur
    return ans
```

2.5 离散化

适用条件：坐标值很大，但实际出现的点较少。

流程：

1. 收集所有有用坐标。
2. 排序去重。
3. 用字典或二分把原坐标映射到较小下标。

```
def compress(vals):
    xs = sorted(set(vals))
    mp = {x: i for i, x in enumerate(xs)}
    return xs, mp
```

2.6 二分查找

左边界模板：寻找第一个满足 $a[i] \geq x$ 的位置。

```
def lower_bound(a, x):
    l, r = 0, len(a)
    while l < r:
        mid = (l + r) // 2
        if a[mid] >= x:
            r = mid
        else:
            l = mid + 1
    return l
```

2.7 二分答案

核心判断：答案具有单调性。即某个值可行，则更大或更小的一侧也可行。

流程：

1. 确定答案范围 $[L, R]$ 。
2. 写 `check(mid)` 判断 mid 是否可行。
3. 根据可行性收缩区间。

适用题型：最小最大值、最大最小值、最短时间、最少代价、容量问题。

```
def binary_answer_left(lo, hi, check):
    # find minimum x such that check(x) is True
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def binary_answer_right(lo, hi, check):
    # find maximum x such that check(x) is True
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if check(mid):
            lo = mid
        else:
            hi = mid - 1
    return lo
```

3 双指针、滑动窗口与单调结构

3.1 双指针

适用题型：有序数组两数和、区间满足某条件、去重、合并。

```
def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return l, r
        elif s < target:
            l += 1
        else:
            r -= 1
    return -1, -1
```

3.2 滑动窗口

适用条件：窗口扩大或缩小时，性质能被维护，常见于正数数组、字符计数。

```
def min_len_at_least_s(a, S):
    ans = 10 ** 18
    cur = 0
    l = 0
    for r, x in enumerate(a):
        cur += x
        while cur >= S:
            ans = min(ans, r - l + 1)
            cur -= a[l]
            l += 1
    return ans if ans < 10 ** 18 else -1
```

3.3 单调栈

适用题型：下一个更大/更小元素、柱状图最大矩形、贡献法计算。

```
def next_greater(a):
    n = len(a)
    ans = [-1] * n
    st = []
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            ans[st.pop()] = i
        st.append(i)
    return ans
```

3.4 单调队列

适用题型：固定长度滑动窗口最大值/最小值，DP 优化。

```
from collections import deque

def sliding_window_max(a, k):
    q = deque()
```

```
ans = []
for i, x in enumerate(a):
    while q and q[0] <= i - k:
        q.popleft()
    while q and a[q[-1]] <= x:
        q.pop()
    q.append(i)
    if i >= k - 1:
        ans.append(a[q[0]])
return ans
```

4 贪心

4.1 贪心的判断方法

常见可贪心信号：

- 每一步局部选择不会影响后续最优性。
- 可以排序后选择。
- 可以用交换论证证明。
- 题目要求最大数量、最早结束、最小代价。

4.2 区间调度

选择最多不重叠区间，按右端点升序排序。

```
def max_non_overlap(intervals):
    intervals.sort(key=lambda x: x[1])
    ans = 0
    last_end = -10 ** 30
    for l, r in intervals:
        if l >= last_end:
            ans += 1
            last_end = r
    return ans
```

4.3 区间合并

```
def merge_intervals(intervals):
    intervals.sort()
    res = []
    for l, r in intervals:
        if not res or l > res[-1][1]:
            res.append([l, r])
        else:
            res[-1][1] = max(res[-1][1], r)
    return res
```

5 数据结构

5.1 堆

适用题型：动态最小值、动态最大值、Top-K、合并有序序列、Dijkstra。

```
from heapq import heappush, heappop

pq = []
heappush(pq, (cost, node))
cost, node = heappop(pq)

# max heap
heappush(pq, -x)
x = -heappop(pq)
```

5.2 懒删除堆

```
from heapq import heappush, heappop
from collections import Counter

class LazyHeap:
    def __init__(self):
        self.h = []
        self.del_cnt = Counter()

    def push(self, x):
        heappush(self.h, x)

    def erase(self, x):
        self.del_cnt[x] += 1

    def _clean(self):
        while self.h and self.del_cnt[self.h[0]]:
            x = heappop(self.h)
            self.del_cnt[x] -= 1

    def top(self):
        self._clean()
        return self.h[0] if self.h else None

    def pop(self):
        self._clean()
        return heappop(self.h)
```

5.3 并查集

适用题型：连通性、合并集合、最小生成树、动态加边。

```
class DSU:
    def __init__(self, n):
```

```

self.fa = list(range(n))
self.sz = [1] * n

def find(self, x):
    while self.fa[x] != x:
        self.fa[x] = self.fa[self.fa[x]]
        x = self.fa[x]
    return x

def union(self, a, b):
    ra, rb = self.find(a), self.find(b)
    if ra == rb:
        return False
    if self.sz[ra] < self.sz[rb]:
        ra, rb = rb, ra
    self.fa[rb] = ra
    self.sz[ra] += self.sz[rb]
    return True

```

5.4 树状数组

功能：单点修改、前缀查询。也可配合差分做区间加。

```

class BIT:
    def __init__(self, n):
        self.n = n
        self.t = [0] * (n + 1)

    def add(self, i, v):
        i += 1
        while i <= self.n:
            self.t[i] += v
            i += i & -i

    def sum(self, i):
        # sum of [0, i]
        i += 1
        res = 0
        while i > 0:
            res += self.t[i]
            i -= i & -i
        return res

    def range_sum(self, l, r):
        if r < l:
            return 0
        return self.sum(r) - (self.sum(l - 1) if l else 0)

```

5.5 线段树

适用题型：区间查询、区间修改、动态维护区间和/最值/最大子段和等。

```
class SegTree:
    def __init__(self, a):
        self.n = len(a)
        self.sum = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(1, 0, self.n - 1, a)

    def _build(self, o, l, r, a):
        if l == r:
            self.sum[o] = a[l]
            return
        m = (l + r) // 2
        self._build(o * 2, l, m, a)
        self._build(o * 2 + 1, m + 1, r, a)
        self.sum[o] = self.sum[o * 2] + self.sum[o * 2 + 1]

    def _apply(self, o, l, r, v):
        self.sum[o] += (r - l + 1) * v
        self.lazy[o] += v

    def _push(self, o, l, r):
        if self.lazy[o]:
            m = (l + r) // 2
            v = self.lazy[o]
            self._apply(o * 2, l, m, v)
            self._apply(o * 2 + 1, m + 1, r, v)
            self.lazy[o] = 0

    def add(self, L, R, v):
        self._add(1, 0, self.n - 1, L, R, v)

    def _add(self, o, l, r, L, R, v):
        if L <= l and r <= R:
            self._apply(o, l, r, v)
            return
        self._push(o, l, r)
        m = (l + r) // 2
        if L <= m:
            self._add(o * 2, l, m, L, R, v)
        if R > m:
            self._add(o * 2 + 1, m + 1, r, L, R, v)
        self.sum[o] = self.sum[o * 2] + self.sum[o * 2 + 1]

    def query(self, L, R):
        return self._query(1, 0, self.n - 1, L, R)

    def _query(self, o, l, r, L, R):
```

```

if L <= 1 and r <= R:
    return self.sum[o]
self._push(o, l, r)
m = (l + r) // 2
res = 0
if L <= m:
    res += self._query(o * 2, l, m, L, R)
if R > m:
    res += self._query(o * 2 + 1, m + 1, r, L, R)
return res

```

5.6 ST 表

适用题型：静态区间最值查询，不能修改。

```

from math import log2

class SparseTable:
    def __init__(self, a, func=min):
        self.func = func
        n = len(a)
        K = n.bit_length()
        self.st = [a[:]]
        j = 1
        while (1 << j) <= n:
            prev = self.st[-1]
            length = 1 << (j - 1)
            cur = [
                func(prev[i], prev[i + length])
                for i in range(n - (1 << j) + 1)
            ]
            self.st.append(cur)
            j += 1

    def query(self, l, r):
        # inclusive [l, r]
        k = (r - l + 1).bit_length() - 1
        return self.func(self.st[k][l], self.st[k][r - (1 << k) + 1])

```

6 图论

6.1 建图模板

```

def build_graph(n, edges, directed=False):
    g = [[] for _ in range(n)]
    for u, v in edges:
        g[u].append(v)
        if not directed:
            g[v].append(u)

```

```
return g
```

6.2 BFS

适用题型：无权最短路、网格最短路、层次遍历。

```
from collections import deque

def bfs(g, s):
    n = len(g)
    dist = [-1] * n
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

6.3 DFS

适用题型：连通块、树 DP、回溯、拓扑辅助。

```
def dfs_iter(g, s):
    n = len(g)
    vis = [False] * n
    st = [s]
    vis[s] = True
    order = []
    while st:
        u = st.pop()
        order.append(u)
        for v in g[u]:
            if not vis[v]:
                vis[v] = True
                st.append(v)
    return order
```

6.4 拓扑排序

适用题型：有向无环图、课程安排、任务依赖、DAG DP。

```
from collections import deque

def topo_sort(n, edges):
    g = [[] for _ in range(n)]
    indeg = [0] * n
    for u, v in edges:
        g[u].append(v)
```

```

    indeg[v] += 1

q = deque(i for i in range(n) if indeg[i] == 0)
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in g[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)

return order if len(order) == n else []

```

6.5 Dijkstra

适用条件：边权非负。

```

from heapq import heappush, heappop

def dijkstra(n, g, s):
    # g[u] = [(v, w), ...]
    INF = 10 ** 30
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heappop(pq)
        if d != dist[u]:
            continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heappush(pq, (nd, v))
    return dist

```

6.6 Floyd

适用题型：点数较小的任意两点最短路，通常 $n \leq 400$ 。

```

def floyd(dist):
    n = len(dist)
    for k in range(n):
        for i in range(n):
            dik = dist[i][k]
            for j in range(n):
                if dik + dist[k][j] < dist[i][j]:
                    dist[i][j] = dik + dist[k][j]
    return dist

```

6.7 最小生成树 Kruskal

```
def kruskal(n, edges):
    # edges = [(w, u, v), ...]
    dsu = DSU(n)
    edges.sort()
    total = 0
    cnt = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            total += w
            cnt += 1
    return total if cnt == n - 1 else None
```

7 动态规划

7.1 DP 设计四步

1. 定义状态： $dp[i]$ 或 $dp[i][j]$ 表示什么。
2. 确定转移：从哪些旧状态推到新状态。
3. 初始化：边界状态是什么。
4. 确定答案：是 $dp[n]$ 、最大值、最小值还是求和。

7.2 一维线性 DP

```
def max_subarray(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

7.3 最长上升子序列

$O(n^2)$ 或 $O(n \log n)$

```
from bisect import bisect_left

def lis_length(a):
    d = []
    for x in a:
        i = bisect_left(d, x)
        if i == len(d):
            d.append(x)
        else:
            d[i] = x
    return len(d)
```

7.4 01 背包

适用题型：每个物品最多选一次。

```
def zero_one_knapsack(items, W):
    # items = [(weight, value), ...]
    dp = [0] * (W + 1)
    for w, v in items:
        for cap in range(W, w - 1, -1):
            dp[cap] = max(dp[cap], dp[cap - w] + v)
    return dp[W]
```

7.5 完全背包

适用题型：每个物品可以选无限次。

```
def complete_knapsack(items, W):
    dp = [0] * (W + 1)
    for w, v in items:
        for cap in range(w, W + 1):
            dp[cap] = max(dp[cap], dp[cap - w] + v)
    return dp[W]
```

7.6 区间 DP

适用题型：合并石子、括号匹配、区间划分。

```
def interval_dp(a):
    n = len(a)
    INF = 10 ** 30
    dp = [[0] * n for _ in range(n)]
    pre = [0]
    for x in a:
        pre.append(pre[-1] + x)

    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            dp[l][r] = INF
            total = pre[r + 1] - pre[l]
            for k in range(l, r):
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + total)
    return dp[0][n - 1]
```

7.7 树形 DP

适用题型：树上最大独立集、树上路径、以子树为单位统计。

```
import sys
sys.setrecursionlimit(10 ** 7)

def tree_dp(n, g, val):
```

```

dp0 = [0] * n
dp1 = [0] * n

def dfs(u, p):
    dp1[u] = val[u]
    for v in g[u]:
        if v == p:
            continue
        dfs(v, u)
        dp0[u] += max(dp0[v], dp1[v])
        dp1[u] += dp0[v]

dfs(0, -1)
return max(dp0[0], dp1[0])

```

7.8 状态压缩 DP

适用条件: n 较小, 一般 $n \leq 20$ 。

```

def tsp(dist):
    n = len(dist)
    INF = 10 ** 30
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for u in range(n):
            if dp[mask][u] >= INF:
                continue
            for v in range(n):
                if mask >> v & 1:
                    continue
                nm = mask | (1 << v)
                dp[nm][v] = min(dp[nm][v], dp[mask][u] + dist[u][v])
    return min(dp[(1 << n) - 1])

```

7.9 数位 DP

适用题型: 统计 $[0, n]$ 内满足某种数字性质的数。

```

from functools import lru_cache

def count_numbers(n):
    s = str(n)

    @lru_cache(None)
    def dfs(pos, tight, started, state):
        if pos == len(s):
            return 1 if started else 0

        limit = int(s[pos]) if tight else 9
        ans = 0

```

```

for d in range(limit + 1):
    ntight = tight and (d == limit)
    nstarted = started or d != 0
    nstate = state
    if nstarted:
        # update nstate here
        pass
    ans += dfs(pos + 1, ntight, nstarted, nstate)
return ans

return dfs(0, True, False, 0)

```

8 数学与数论

8.1 最大公约数与最小公倍数

```

from math import gcd

def lcm(a, b):
    return a // gcd(a, b) * b

```

8.2 快速幂

```

def qpow(a, b, mod):
    res = 1
    a %= mod
    while b:
        if b & 1:
            res = res * a % mod
        a = a * a % mod
        b >>= 1
    return res

```

8.3 逆元

当模数 MOD 是质数时:

$$a^{-1} \equiv a^{MOD-2} \pmod{MOD}.$$

```

def inv(a, mod):
    return pow(a, mod - 2, mod)

```

8.4 组合数预处理

```

def prepare_comb(n, mod):
    fac = [1] * (n + 1)
    ifac = [1] * (n + 1)
    for i in range(1, n + 1):

```

```
    fac[i] = fac[i - 1] * i % mod
    ifac[n] = pow(fac[n], mod - 2, mod)
    for i in range(n, 0, -1):
        ifac[i - 1] = ifac[i] * i % mod

    def C(n, k):
        if k < 0 or k > n:
            return 0
        return fac[n] * ifac[k] % mod * ifac[n - k] % mod

    return C
```

8.5 素数筛

```
def sieve(n):
    is_prime = [True] * (n + 1)
    if n >= 0:
        is_prime[0] = False
    if n >= 1:
        is_prime[1] = False
    primes = []
    for i in range(2, n + 1):
        if is_prime[i]:
            primes.append(i)
            step = i
            start = i * i
            if start <= n:
                for j in range(start, n + 1, step):
                    is_prime[j] = False
    return primes, is_prime
```

8.6 质因数分解

```
def factorize(x):
    res = []
    d = 2
    while d * d <= x:
        if x % d == 0:
            cnt = 0
            while x % d == 0:
                x //= d
                cnt += 1
            res.append((d, cnt))
        d += 1 if d == 2 else 2
    if x > 1:
        res.append((x, 1))
    return res
```

8.7 矩阵快速幂

适用题型：线性递推、斐波那契变形、固定状态转移重复 k 次。

```
def mat_mul(A, B, mod):
    n, m, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(n)]
    for i in range(n):
        for k in range(m):
            if A[i][k]:
                aik = A[i][k]
                for j in range(p):
                    C[i][j] = (C[i][j] + aik * B[k][j]) % mod
    return C

def mat_pow(A, e, mod):
    n = len(A)
    R = [[0] * n for _ in range(n)]
    for i in range(n):
        R[i][i] = 1
    while e:
        if e & 1:
            R = mat_mul(R, A, mod)
        A = mat_mul(A, A, mod)
        e >>= 1
    return R
```

9 字符串算法

9.1 KMP

适用题型：模式串匹配、求最短循环节、前后缀关系。

```
def prefix_function(s):
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        j = pi[i - 1]
        while j and s[i] != s[j]:
            j = pi[j - 1]
        if s[i] == s[j]:
            j += 1
        pi[i] = j
    return pi

def kmp_find(text, pat):
    if not pat:
        return list(range(len(text) + 1))
    pi = prefix_function(pat)
    ans = []
    j = 0
```

```
for i, ch in enumerate(text):
    while j and ch != pat[j]:
        j = pi[j - 1]
    if ch == pat[j]:
        j += 1
    if j == len(pat):
        ans.append(i - len(pat) + 1)
        j = pi[j - 1]
return ans
```

9.2 Trie 字典树

适用题型：前缀统计、字符串集合、异或最大值。

```
class Trie:
    def __init__(self):
        self.ch = []
        self.end = []
        self.ch.append({})
        self.end.append(0)

    def insert(self, s):
        u = 0
        for c in s:
            if c not in self.ch[u]:
                self.ch[u][c] = len(self.ch)
                self.ch.append({})
                self.end.append(0)
            u = self.ch[u][c]
        self.end[u] += 1

    def search(self, s):
        u = 0
        for c in s:
            if c not in self.ch[u]:
                return 0
            u = self.ch[u][c]
        return self.end[u]
```

9.3 字符串哈希

```
class StringHash:
    def __init__(self, s, base=911382323, mod=(1 << 61) - 1):
        self.mod = mod
        self.base = base
        n = len(s)
        self.h = [0] * (n + 1)
        self.p = [1] * (n + 1)
        for i, ch in enumerate(s):
            self.h[i + 1] = (self.h[i] * base + ord(ch)) % mod
```

```

        self.p[i + 1] = self.p[i] * base % mod

    def get(self, l, r):
        # [l, r)
        return (self.h[r] - self.h[l] * self.p[r - l]) % self.mod

```

9.4 Manacher

适用题型：最长回文子串、统计回文半径。

```

def manacher(s):
    t = "#" + "#".join(s) + "#"
    n = len(t)
    p = [0] * n
    center = right = 0
    best = 0
    for i in range(n):
        mirror = 2 * center - i
        if i < right:
            p[i] = min(right - i, p[mirror])
        while i - p[i] - 1 >= 0 and i + p[i] + 1 < n and t[i - p[i] - 1] == t[i + p[i] + 1]:
            p[i] += 1
        if i + p[i] > right:
            center, right = i, i + p[i]
        best = max(best, p[i])
    return best

```

10 搜索、回溯与剪枝

10.1 回溯模板

适用题型：排列、组合、棋盘、子集枚举、小规模构造。

```

def backtrack(path, used, nums, ans):
    if len(path) == len(nums):
        ans.append(path[:])
        return
    for i, x in enumerate(nums):
        if used[i]:
            continue
        used[i] = True
        path.append(x)
        backtrack(path, used, nums, ans)
        path.pop()
        used[i] = False

```

10.2 Meet in the Middle

适用条件： n 大约 30 到 44，直接 2^n 不可行，但 $2^{n/2}$ 可行。

```
from bisect import bisect_right

def subset_sums(a):
    res = [0]
    for x in a:
        res += [y + x for y in res]
    return res

def count_subset_sum_leq(a, limit):
    mid = len(a) // 2
    left = subset_sums(a[:mid])
    right = subset_sums(a[mid:])
    right.sort()
    ans = 0
    for x in left:
        ans += bisect_right(right, limit - x)
    return ans
```

11 位运算

11.1 常用技巧

表达式	含义
$x \& 1$	判断奇偶
$x \gg k \& 1$	判断第 k 位是否为 1
$x \mid (1 \ll k)$	把第 k 位置 1
$x \& \sim(1 \ll k)$	把第 k 位置 0
$x \wedge (1 \ll k)$	翻转第 k 位
$x \& -x$	取最低位的 1
<code>x.bit_count()</code>	统计 1 的个数

11.2 枚举子集

```
def enumerate_submasks(mask):
    sub = mask
    res = []
    while sub:
        res.append(sub)
        sub = (sub - 1) & mask
    res.append(0)
    return res
```

12 扫描线与区间覆盖

12.1 最大重叠区间数量

适用题型：会议室数量、同时在线人数、区间覆盖层数。

```
def max_overlap(intervals):
    events = []
    for l, r in intervals:
        events.append((l, 1))
        events.append((r, -1))
    events.sort(key=lambda x: (x[0], x[1]))
    cur = ans = 0
    for _, delta in events:
        cur += delta
        ans = max(ans, cur)
    return ans
```

12.2 整数闭区间覆盖

若区间是闭区间 $[l, r]$ ，可把结束事件放在 $r + 1$ 。

```
def max_overlap_closed(intervals):
    diff = []
    for l, r in intervals:
        diff.append((l, 1))
        diff.append((r + 1, -1))
    diff.sort()
    cur = ans = 0
    for _, d in diff:
        cur += d
        ans = max(ans, cur)
    return ans
```

13 博弈论

13.1 Nim 游戏

若若干堆石子，每次选择一堆拿任意正数个，则：

$$a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0$$

为先手必胜。

```
def nim_winner(piles):
    x = 0
    for v in piles:
        x ^= v
    return "First" if x else "Second"
```

13.2 SG 函数

```
def mex(s):
    g = 0
    while g in s:
        g += 1
    return g

def sg_values(n, moves):
    sg = [0] * (n + 1)
    for i in range(1, n + 1):
        reach = set()
        for m in moves:
            if i >= m:
                reach.add(sg[i - m])
        sg[i] = mex(reach)
    return sg
```

14 常见综合题型

14.1 最值 + 可行性

题型：给一个答案 x ，问能否在限制内完成。

套路：二分答案 + 贪心/前缀/图搜索判断。

答案单调 $\Rightarrow$ 二分

14.2 区间修改 + 区间查询

题型：多次修改数组区间，并查询区间和/最值。

套路：

- 只有最终结果：差分。
- 单点修改 + 区间和：树状数组。
- 区间修改 + 区间查询：线段树。

14.3 连通块问题

题型：网格岛屿、朋友关系、加边后连通性。

套路：

- 静态图：DFS/BFS。
- 动态合并：并查集。
- 带权边最小连接：Kruskal。

14.4 计数取模

题型：方案数、排列数、路径数。

套路：

- 有明显递推：DP。
- 选择若干元素：组合数。
- 大指数：快速幂。
- 线性递推次数很大：矩阵快速幂。

14.5 字符串匹配

题型：判断模式串出现次数、最长公共前缀、回文。

套路：

- 单模式匹配：KMP。
- 多字符串前缀：Trie。
- 子串快速比较：字符串哈希。
- 回文子串：Manacher。

15 比赛实用技巧

15.1 读题顺序

1. 先看数据范围。
2. 再看是否多组测试。
3. 找关键词：区间、路径、取模、最小最大、方案数。
4. 先写暴力，判断瓶颈。
5. 用对应算法优化瓶颈。

15.2 Python 常见坑

- 递归深度：树 DFS 前设置 `sys.setrecursionlimit`。
- 大输入：使用 `sys.stdin.readline`。
- 取模：每次乘法或加法后及时取模。
- 浮点二分：控制迭代次数，如 80 次。
- 多组测试：注意清空全局数组。
- 下标：明确 0-index 还是 1-index。

15.3 浮点二分模板

```
def float_binary(lo, hi, check):  
    for _ in range(80):  
        mid = (lo + hi) / 2  
        if check(mid):  
            hi = mid  
        else:  
            lo = mid  
    return (lo + hi) / 2
```

15.4 常用取模写法

```
MOD = 10 ** 9 + 7  
  
x = (x + y) % MOD  
x = x * y % MOD  
x = (x - y) % MOD
```

16 赛前复习清单

- 会写快速输入模板。
- 会写二分答案，并能判断单调性。
- 会写前缀和、差分、离散化。
- 会写并查集、树状数组、线段树。
- 会写 BFS、Dijkstra、拓扑排序。
- 会写 01 背包、完全背包、LIS、区间 DP。
- 会写快速幂、组合数、素数筛。
- 会写 KMP、Trie、字符串哈希。
- 熟悉常见边界：空数组、单元素、负数、闭区间、取模。

17 最后的选型口诀

- 看到“多次区间求和”，先想前缀和。
- 看到“多次区间修改”，先想差分或线段树。
- 看到“最大最小/最小最大”，先想二分答案。
- 看到“连通性”，先想并查集或 DFS。
- 看到“最短路”，先看边权：无权 BFS，非负 Dijkstra。
- 看到“方案数取模”，先想 DP 或组合数学。

- 看到“字符串匹配”，先想 KMP 或哈希。
- 看到“坐标巨大但数量少”，先想离散化。
- 看到“ $n \leq 20$ ”，先想状态压缩。
- 看到“两人轮流操作”，先想博弈论。